DESIGN DOCUMENT

AUDIO FORMAT

Document History:

S.No	Version No.	Author	Section(s) Affected	Change(s) made	Approved Date	Approved by
1	1.0	Nithishkumar.T	All	Initial		
2	1.1	Nithishkumar.T	Low Level Design	Design image		
3	1.2	Nithishkumar T	Low Level Design	Design description		

Table of Contents

1. INTRODUCTION

This project is about the conversion of the given audio sample input into various formats of data. Audio data often needs to be transferred between different representations, such as block-based and interleaved formats, depending on the application or processing requirements. Block-based formats store each channel's samples in separate, contiguous blocks, while interleaved formats arrange the samples of all channels sequentially within the same buffer. Efficient conversion between these formats is crucial for optimizing memory usage and computational performance in real-time audio applications. The goal of this project is to design a API that efficiently handles these format conversions, ensuring minimal latency and resource consumption in diverse environments, from embedded systems to high-performance audio workstations.

2. PROBLEM STATEMENT

The problem statement is about developing a single API that supports to move the audio samples from one buffer to another. For that D19API should support multiple formats and the below mentioned format needs to support

- Block to Block
- Block to Interleaved
- Interleaved to Interleaved
- Interleaved to Block

It should have different input arguments like, block size, interleaved stride and the API should be an optimized one in terms of MIPS and Memory

3. OBJECTIVE

The main objective of this is to implement the audio format conversion from the sample audio input with different forms of conversion. The conversion involves Block to Block, Block to Interleaved, Interleaved to Interleaved and Interleaved to Block. For each conversion the previous output data is got to the input of the conversion, and the format needs to be changed.

- **Block to Block**

  This operation transfers audio samples from one block-based buffer to another, preserving the block structure. Each block contains

the samples of a single channel, and the operation ensures that the data is efficiently moved while maintaining the original layout.

- **Block to Interleaved**

  In this transformation, audio data stored in a block-based format (where each channel has its own buffer) is converted to an interleaved format (where the samples of all channels are interspersed in a single buffer). This operation requires careful mapping of samples from each channel into a continuous buffer with the correct stride for each channel.

- **Interleaved to Interleaved**

  This conversion allows the transfer of audio data between two interleaved buffers. This can be particularly useful in scenarios where the stride or the number of channels may differ between the two buffers, and the data must be adjusted accordingly. The operation involves ensuring proper reorganization and alignment of the data to match the target buffer's interleaving pattern.

- **Interleaved to Block**

  This operation converts interleaved audio data into a block-based format, where each channel's data is separated into distinct blocks. The samples in the interleaved buffer must be extracted and reallocated to individual block buffers corresponding to each channel.

4. DESIGN PLAN

4.1 SYSTEM OVERVIEW

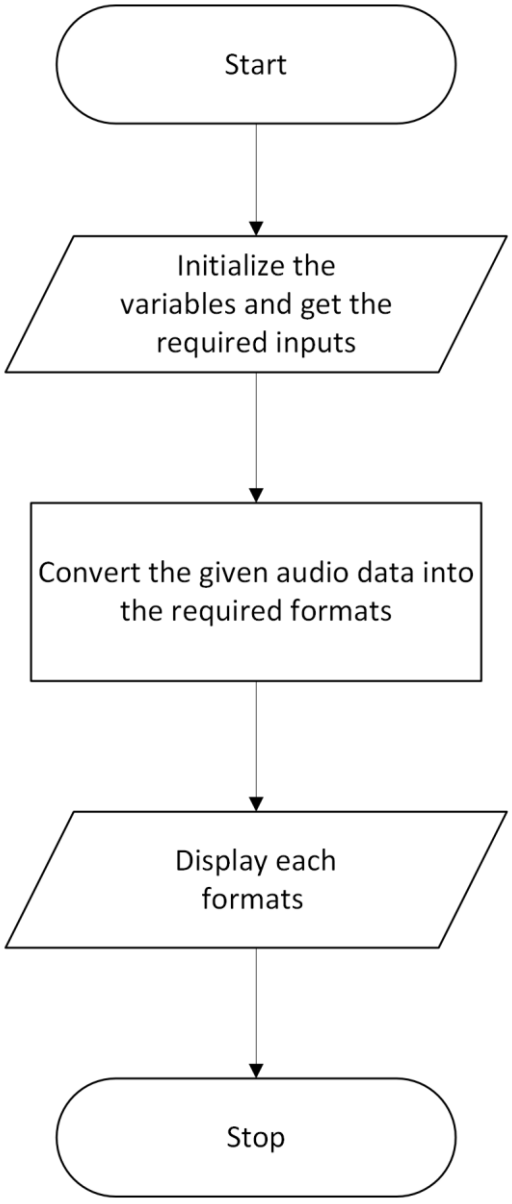
The System changes the input of the audio file from the given sample audio so that we can convert the required formatted files. This will be done by making the identification of the block size and the interleaved stride size which are made by the user that can be available from the source data. The code needs to be optimized in that system and the memory management also viewed. Based on this the optimization code is developed. The working of the conversion is described in the following design.

4.2 DESIGN

4.2.1 HIGH LEVEL DESIGN

In the High-Level design, the block explains the overall way of approaching the given problem statement. The design begins with initializing the necessary variables and gathering required inputs, such as the audio file to

be processed and any preferences for the output format. After the setup, the next step is to convert the input audio data into the desired formats. This involves reading the input audio file, processing the data and then encoding the audio into the specified formats. Once the conversion is complete, the resulting files are displayed or logged, showing details of the format file. Finally, the process ends with the program stopping, potentially saving the converted files and cleaning up any resources. The High-Level Design is attached as follows.



4.2.2 LOW LEVEL DESIGN

In the Low-Level design, each block explains the working way of approaching the given problem statement. In that the input file is transferred and converted based on the required format.

The low-level design outlines a process for transforming data between different formats, specifically focusing on converting between block data format and interleaved data format across four distinct modes.

The process begins by loading an input file, which contains data either in Block Format or Interleaved Format. The system then retrieves two important parameters: the block size (which defines how the data is grouped in blocks) and the interleave stride (which defines the spacing between elements in an interleaved format). These parameters are necessary for the conversion process. Next, the user or system selects a conversion mode, which determines how the data will be transformed. This is handled using a switch-case structure, with four possible modes.

- **Mode 1 (Block Format to Block Format)**
  The system simply reads the data in block format and converts it to another block format, potentially with different block sizes or organization.
- **Mode 2 (Block Format to Interleaved Format)**
  The system converts data from block format to interleaved format by reorganizing the blocks into an interwoven structure, where elements from different blocks are mixed together in a specific order.
- **Mode 3 (Interleaved Format to Interleaved Format)**
  Involves rearranging interleaved data by changing the interleaved stride or the order in which elements are interleaved.
- **Mode 4 (Interleaved Format to Block Format)**
  Interleaved data is transformed back into block format, grouping elements into blocks for easier processing in certain algorithms.

If an invalid conversion is requested, the system will display an error message, such as **"Invalid input conversion"**. Once the conversion is complete, the system will display the newly formatted data, and the process ends. This ensures that the system handles the input file correctly according

to the selected conversion mode, providing the appropriate output or error message when necessary.

Once the conversion is completed it shows the converted output and the process ends. The Low-Level Design is attached as follows.